

STHENOS - Métamodélisation par intelligence artificielle pour la substitution de solveurs par éléments finis en mécanique des structures [Etape 1]

Rayan Chennaoui, Nima Kashani *Étudiants IPSA 4A*,
Spécialité Mécanique / Structures Aéronautiques
rayan.chennaoui@ipsa.fr, nima.kashani@ipsa.fr

STHENOS (du grec ancien $\Sigma\theta\acute{\epsilon}\nu\omicron\varsigma$, signifiant *la force, la puissance*).

Résumé—L'analyse par éléments finis (FEA) est un pilier du dimensionnement structural en aéronautique, mais son coût computationnel prohibitif limite son intégration dans les boucles d'optimisation itérative. Le projet STHENOS détaille la création d'un métamodèle basé sur l'Intelligence Artificielle (*Surrogate Model*) pour remplacer le solveur MSC Nastran (SOL 101) dans l'étude d'une poutre 3D encastree-libre maillée en éléments volumiques HEXA20. En s'appuyant sur un plan d'expériences généré par *Latin Hypercube Sampling* (LHS) sur deux paramètres ($L \in [500, 2500]$ mm, $F \in [1000, 16000]$ N), un jeu de données de 296 points a été constitué via un pipeline automatisé de génération de fichiers `.bdf`, d'exécution Nastran en mode batch et de parsing des fichiers `.f06` par expressions régulières. Trois architectures de Machine Learning ont été développées avec la bibliothèque scikit-learn : Régression Polynomiale avec régularisation Ridge, Forêt Aléatoire et Réseau de Neurones Multi-Couches (MLP). Le MLP surpasse les autres modèles avec un coefficient de détermination $R^2 > 0.999$, une erreur absolue moyenne MAE ≈ 3 mm sur la flèche et ≈ 6.4 MPa sur la contrainte de Von Mises, le tout pour un facteur de gain de vitesse (*Speed-Up*) de $\times 50\,000$ par rapport au solveur EF. Un démonstrateur temps réel est déployé sous Streamlit.

Abstract—Finite Element Analysis (FEA) is a cornerstone of structural sizing in aeronautics, but its prohibitive computational cost limits its integration into iterative optimization loops. The STHENOS project details the creation of an AI-based surrogate model to replace the MSC Nastran solver (SOL 101) in the study of a 3D cantilever beam meshed with HEXA20 volumetric elements. Based on a Design of Experiments generated by Latin Hypercube Sampling (LHS) over two parameters ($L \in [500, 2500]$ mm, $F \in [1000, 16000]$ N), a dataset of 296 points was built via an automated pipeline for `.bdf` generation, batch Nastran execution, and `.f06` parsing via regular expressions. Three Machine Learning architectures were developed using scikit-learn : Polynomial Ridge Regression, Random Forest, and Multi-Layer Perceptron (MLP). The MLP achieves $R^2 > 0.999$, MAE ≈ 3 mm and ≈ 6.4 MPa, with a Speed-Up factor of $\times 50,000$ over FEA. A real-time demonstrator is deployed with Streamlit.

I. INTRODUCTION & CONTEXTE AÉRONAUTIQUE

A. Le défi du dimensionnement structural

Le dimensionnement structural est le processus par lequel l'ingénieur aéronautique s'assure qu'une pièce — aile, fuselage, longeron, cadre — résiste aux chargements auxquels elle sera soumise en vol, au décollage, à l'atterrissage, etc. Pour ce faire, il doit connaître, pour chaque point du solide, les **contraintes mécaniques** (les forces internes par unité de surface, en Pa ou MPa) et les **déplacements** (la déformation de la structure, en mm).

Pendant des décennies, cette tâche s'est appuyée exclusivement sur des formules analytiques issues de la résistance des matériaux (RDM) et de la théorie des plaques. Mais ces formules, aussi élégantes soient-elles, ne valent que pour des géométries simples : une poutre droite à section constante, une plaque mince, une coque cylindrique.

Un avion moderne, en revanche, présente des géométries d'une complexité extraordinaire. La seule approche universellement applicable est la **Méthode des Éléments Finis** (MEF ou FEA), implémentée dans des codes comme MSC Nastran, Abaqus ou ANSYS.

B. Le coût prohibitif de la MEF

La MEF discrétise le domaine continu d'un solide en milliers, voire en millions de petits blocs appelés **éléments**. Chaque nœud connectant ces éléments se voit attribuer des inconnues (déplacements), et les équations de la physique permettent d'écrire un gigantesque système linéaire $[K]\{U\} = \{F\}$. La résolution de ce système est l'opération fondamentale, et son coût est de l'ordre $\mathcal{O}(n^3)$ où n est le nombre de degrés de liberté.

Pour un modèle d'avion complet comportant plusieurs millions de nœuds, chaque résolution peut prendre de plusieurs minutes à plusieurs heures. Dans un processus d'optimisation de forme ou de topologie, ce solveur peut être appelé des milliers de fois. Il devient alors un **verrou computationnel**.

C. La solution : les Surrogate Models

L'idée du *Surrogate Model* (ou métamodèle) est de remplacer ce solveur coûteux par une fonction mathématique apprise sur un ensemble d'exemples préalablement calculés. Cette

approche ne repose plus sur la physique interne du solveur, mais sur la reconnaissance de patterns dans les données.

Le présent projet constitue une implémentation de bout en bout de cette philosophie, de la génération des données Nastran jusqu'au déploiement d'une interface web interactive.

II. RAPPELS DE MÉCANIQUE DES MILIEUX CONTINUS

Cette section est un préambule théorique indispensable. Elle pose les fondements de la physique que notre IA devra imiter.

A. Qu'est-ce qu'un milieu continu ?

Un **milieu continu** est une idéalisation mathématique de la matière. Au lieu de modéliser chaque atome ou molécule individuellement, on suppose que la matière est répartie de manière continue dans l'espace. Cette hypothèse est valide dès lors que l'échelle d'observation est grande devant l'échelle moléculaire, ce qui est le cas en mécanique des structures (où l'on parle de millimètres à mètres).

B. Le tenseur des contraintes

Lorsqu'un solide est soumis à des forces, des efforts internes apparaissent. Pour les caractériser, on coupe mentalement le solide par un plan et on regarde les forces que la partie gauche exerce sur la partie droite.

Ces forces internes s'expriment par unité de surface et définissent le **vecteur-contrainte** $\mathbf{T}$. Sa décomposition dans le repère cartésien (x, y, z) fait intervenir 9 composantes, regroupées dans le **tenseur de Cauchy des contraintes** :

$$[\boldsymbol{\sigma}] = \begin{pmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz} \end{pmatrix} \quad (1)$$

Les termes diagonaux $(\sigma_{xx}, \sigma_{yy}, \sigma_{zz})$ sont les **contraintes normales** (traction/compression). Les termes hors-diagonaux $(\sigma_{xy}, \text{etc.})$ sont les **contraintes de cisaillement**. Par symétrie du tenseur $(\sigma_{ij} = \sigma_{ji})$, seules 6 composantes sont indépendantes.

C. La contrainte de Von Mises σ_{VM}

Comparer un état de contrainte complexe (avec normales et cisaillements) à la limite élastique d'un matériau testée en traction simple est un problème non trivial. Le critère de **Von Mises** (1913) fournit une réponse élégante : il définit une contrainte équivalente scalaire σ_{VM} à partir du déviateur du tenseur des contraintes.

En termes de contraintes principales $\sigma_1, \sigma_2, \sigma_3$:

$$\sigma_{VM} = \sqrt{\frac{(\sigma_1 - \sigma_2)^2 + (\sigma_2 - \sigma_3)^2 + (\sigma_3 - \sigma_1)^2}{2}} \quad (2)$$

La condition de rupture plastique est simplement $\sigma_{VM} \geq \sigma_e$ où σ_e est la limite élastique du matériau. C'est **cette grandeur** que Nastran calcule et que notre IA apprend à prédire.

D. Le tenseur des déformations

Sous l'effet des contraintes, le solide se déforme. La déformation locale est quantifiée par le **tenseur des petites déformations** $\boldsymbol{\varepsilon}$, défini à partir du champ de déplacement $\mathbf{u}(x, y, z)$ par :

$$\varepsilon_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right) \quad (3)$$

Les composantes diagonales ε_{ii} représentent les allongements relatifs (dilatations), et les composantes hors-diagonale les glissements angulaires.

E. La loi de Hooke généralisée (comportement élastique)

Pour un matériau élastique linéaire et isotrope (hypothèse valide pour les métaux en petites déformations), la relation entre contraintes et déformations est la **loi de Hooke généralisée** :

$$\sigma_{ij} = \lambda \varepsilon_{kk} \delta_{ij} + 2\mu \varepsilon_{ij} \quad (4)$$

où λ et μ sont les **coefficients de Lamé** liés au module de Young E et au coefficient de Poisson ν par :

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{E}{2(1+\nu)} \quad (5)$$

C'est l'équation constitutive du matériau. Elle ferme le système d'équations de la MMC.

F. Les équations d'équilibre de Navier

En combinant la loi de Hooke, la définition des déformations et les équations d'équilibre local $(\nabla \cdot \boldsymbol{\sigma} + \mathbf{f} = \mathbf{0})$, on obtient les **équations de Navier** qui gouvernent le champ de déplacement $\mathbf{u}$:

$$(\lambda + \mu) \nabla(\nabla \cdot \mathbf{u}) + \mu \nabla^2 \mathbf{u} + \mathbf{f} = \mathbf{0} \quad (6)$$

C'est ce système d'équations aux dérivées partielles (EDP), couplé à des conditions aux limites (encastrement, forces appliquées), que la MEF résout numériquement.

III. MODÈLE PHYSIQUE : POUTRE D'EULER-BERNOULLI ET HEXA20

Cette partie explique comment on calcule, avec des formules mathématiques, la façon dont une poutre plie et se casse sous l'effet d'une force, un peu comme quand on appuie sur le bout d'une règle tenue à une extrémité.

A. La théorie des poutres d'Euler-Bernoulli

La théorie des poutres d'Euler-Bernoulli (EBT) est une simplification 1D des équations de Navier, valable lorsque la longueur L est grande devant les dimensions de la section transversale. Ses hypothèses sont :

- H1.** Les sections droites restent planes et perpendiculaires à la fibre neutre après déformation.
- H2.** Les déformations sont de petite amplitude.
- H3.** Le matériau est élastique, linéaire, et isotrope.

Sous ces hypothèses, et pour une poutre encastree à une extrémité $(x = 0)$ et soumise à une force concentrée F à

l'autre ($x = L$), on peut dériver analytiquement les équations suivantes.

L'équation différentielle de la déformée $v(x)$ est :

$$EI \frac{d^4 v}{dx^4} = q(x) \quad (7)$$

Pour un effort ponctuel F en bout, $q(x) = 0$ sur $[0, L[$ et l'intégration directe donne le profil :

$$v(x) = \frac{F}{6EI} (3Lx^2 - x^3) \quad (8)$$

La **flèche maximale** à l'extrémité libre ($x = L$) est donc :

$$\delta_{\max} = \frac{FL^3}{3EI} \quad (9)$$

Le moment fléchissant $M_f(x) = F(L - x)$ est maximal en pied d'encastrement ($x = 0$). La **contrainte maximale de flexion** (en surface, à distance $h/2$ de la fibre neutre) est :

$$\sigma_{\max} = \frac{M_{f,\max} \cdot (h/2)}{I_z} = \frac{F \cdot L \cdot (h/2)}{I_z} \quad (10)$$

où $I_z = bh^3/12$ est le moment d'inertie de la section rectangulaire de largeur b et hauteur h .

B. Limites de la théorie EBT et passage au modèle 3D HEXA20

Cette formule simple ne capture pas tous les détails de la réalité (torsion, zones fragiles près des fixations). On utilise donc un modèle 3D plus précis, où la poutre est découpée en milliers de petits blocs, pour être plus fidèle à ce qui se passe réellement.

Les équations (9) et (10) sont remarquablement simples, mais elles ignorent des phénomènes réels importants :

- Le cisaillement transversal (non pris en compte par l'hypothèse H1).
- Les effets de concentration de contraintes à l'encastrement.
- Le gauchissement des sections.

Pour capturer ces effets, notre modèle Nastran utilise des éléments **HEXA20** : des hexaèdres à 20 nœuds (8 nœuds de coin + 12 nœuds de milieu d'arête). Ces éléments d'ordre 2 permettent de représenter des champs de déplacement quadratiques à l'intérieur de chaque élément, offrant une précision nettement supérieure aux éléments linéaires HEXA8. Le format Nastran `.bdf` décrit ce maillage sous la forme de cartes GRID (coordonnées nodales) et CHEXA (connectivités).

C. L'équation matricielle globale de la MEF

Le logiciel Nastran assemble les équations de chacun de ces petits blocs pour former une seule grande équation géante, qu'il résout ensuite pour connaître le comportement de la poutre dans son ensemble.

La méthode des éléments finis repose sur la discrétisation du domaine en éléments volumiques. Pour chaque élément HEXA20 (20 nœuds, soit 60 degrés de liberté), le champ de

déplacement continu est interpolé à partir des déplacements nodaux $\{u_e\}$ via les fonctions de forme quadratiques $[N]$:

$$\mathbf{u}(\mathbf{x}) = [N(\mathbf{x})]\{u_e\} \quad (11)$$

En appliquant l'opérateur différentiel aux fonctions de forme, on définit la matrice cinématique $[B]$, reliant ainsi directement les déformations aux déplacements :

$$\{\varepsilon\} = [B]\{u_e\} \quad (12)$$

La matrice de rigidité élémentaire $[k_e]$ découle du principe des travaux virtuels (un système mécanique est en équilibre si le travail virtuel total de toutes les forces appliquées est nul pour tout déplacement virtuel compatible avec les liaisons du système) et de la loi de Hooke généralisée via la matrice élastique $[D]$:

$$[k_e] = \int_{V_e} [B]^T [D] [B] dV \quad (13)$$

Pour calculer cette intégrale sur un élément réel, on applique un changement de variables vers un cube de référence normalisé $[-1, 1]^3$. La matrice jacobienne $[J]$ assure la correspondance géométrique entre l'élément réel et ce cube virtuel, son déterminant $\det[J]$ traduisant le changement de volume local. Le calcul analytique étant complexe, l'intégrale est approchée numériquement par une quadrature de Gauss-Legendre à 27 points d'intégration ($3 \times 3 \times 3$ points de Gauss ξ_i associés aux poids w_i) :

$$[k_e] \approx \sum_{i=1}^{27} w_i [B(\xi_i)]^T [D] [B(\xi_i)] \det[J(\xi_i)] \quad (14)$$

Pour passer de l'échelle d'un élément à celle de la structure complète, les matrices locales $[k_e]$ sont combinées au sein de la matrice de rigidité globale $[K]$. Les matrices d'aiguillage $[T_e]$ assurent le bon placement des contributions de chaque nœud partagé entre éléments voisins :

$$[K] = \sum_e [T_e]^T [k_e] [T_e] \quad (15)$$

Tant que la structure n'est pas fixée, elle est libre de se déplacer dans l'espace sans se déformer, rendant la matrice $[K]$ non inversible. L'application de l'encastrement à $x = 0$ supprime ces mouvements de corps libre et permet au solveur Nastran (SOL 101) de résoudre le système d'équations linéaire :

$$[K]\{U\} = \{F\} \quad (16)$$

Une fois les déplacements nodaux $\{U\}$ obtenus, le solveur évalue les contraintes aux points de Gauss avant de les extrapoler aux nœuds pour calculer le champ de Von Mises σ_{VM} .

IV. ÉTAT DE L'ART : DU SOLVEUR EF AUX SURROGATE MODELS

A. La révolution Data-Driven Engineering

L'émergence du *Machine Learning* dans les sciences de l'ingénieur au cours de la dernière décennie a ouvert une voie nouvelle : plutôt que de résoudre les équations de la physique, on apprend la relation entrée-sortie à partir d'exemples. Cette

approche, regroupée sous l'appellation *Data-Driven Engineering*, complète (et parfois remplace) les méthodes classiques.

Les métamodèles les plus répandus en mécanique des structures sont :

- **Krigeage (GPR)** : Interpolation bayésienne, offre des estimations d'incertitude. Très performant en faibles dimensions.
- **Machines à Vecteurs de Support (SVM)** : Robuste avec peu de données, mais peu scalable.
- **Réseaux de Neurones (MLP, CNN)** : Scalables, capables de capturer des non-linéarités complexes, actuellement à l'état de l'art.
- **Physics-Informed Neural Networks (PINNs)** : Intègrent les équations de Navier directement dans la fonction de coût. Perspective d'avenir du projet STHENOS.

V. MÉTHODOLOGIE ET PLAN D'EXPÉRIENCES (DOE)

Avant d'entraîner l'IA, il faut lui fournir des exemples. Cette partie explique comment on a choisi les centaines de poutres différentes à calculer pour construire notre base d'exemples.

A. Définition du domaine paramétrique

Dans le but de simplifier le projet, notre système comporte deux paramètres de conception libres :

- La longueur de la poutre : $L \in [500, 2500]$ mm.
- La force appliquée : $F \in [1000, 16000]$ N.

Tous les autres paramètres (section transversale $b \times h = 50 \times 50$ mm, matériau aluminium $E = 70$ GPa, $\nu = 0.3$) sont maintenus constants.

B. L'échantillonnage de Monte Carlo (référence)

La méthode la plus simple consiste à tirer des valeurs de longueur et de force complètement au hasard. Le problème est que certaines combinaisons seront testées plusieurs fois, alors que d'autres ne seront jamais explorées.

L'approche standard pour explorer un espace de paramètres consiste à effectuer un **tirage aléatoire uniforme de Monte Carlo (MC)**. On génère N points de manière totalement indépendante dans chaque dimension. Pour une variable j définie sur l'intervalle physique $[a_j, b_j]$, la coordonnée du i -ième échantillon s'exprime à partir d'une variable aléatoire uniforme $U_{ij} \sim \mathcal{U}(0, 1)$:

$$X_i^j = a_j + (b_j - a_j) \cdot U_{ij} \quad (17)$$

Bien que simple à implémenter, le tirage Monte Carlo brut présente un inconvénient majeur : l'indépendance totale des tirages entraîne une mauvaise répartition spatiale. Des points ont tendance à s'agglutiner dans certaines régions (*clustering*) tout en laissant de vastes zones inexplorées (*gaps*).

Sur le plan statistique, l'erreur d'échantillonnage décroît très lentement, avec un écart-type en $\mathcal{O}(1/\sqrt{N})$, ce qui impose un nombre d'échantillons très élevé pour garantir une couverture représentative. Pour notre plan d'expériences en dimension 2 (L et F), cette dispersion aléatoire est sous-optimale et justifie le recours à une méthode stratifiée.

C. Le Latin Hypercube Sampling (LHS)

Nous utilisons une méthode plus intelligente qui répartit les essais de façon régulière sur toute la plage possible, un peu comme remplir une grille sans laisser de trous, afin d'obtenir des exemples bien représentatifs avec moins de calculs à lancer.

L'échantillonnage par hypercube latin (**LHS**, McKay et al., 1979) est une méthode d'échantillonnage stratifié conçue pour garantir une couverture homogène et sans redondance de l'espace des paramètres, évitant ainsi les défauts de regroupement du tirage aléatoire brut.

Principe algorithmique :

- 1) **Stratification** : Chaque dimension j (de domaine unitaire $[0, 1]$) est découpée en N sous-intervalles disjoints et équiprobables :

$$I_k = \left[\frac{k-1}{N}, \frac{k}{N} \right], \quad k \in \{1, \dots, N\} \quad (18)$$

- 2) **Permutation** : Pour chaque axe j , on génère une permutation aléatoire indépendante π_j de l'ensemble $\{1, \dots, N\}$. Cette étape détermine l'ordre d'association des strates entre les différentes dimensions.

- 3) **Tirage intra-strate** : Dans chaque strate sélectionnée, la coordonnée normalisée du i -ième échantillon est tirée uniformément à l'aide d'une variable $U_{ij} \sim \mathcal{U}(0, 1)$:

$$\tilde{X}_i^j = \frac{\pi_j(i) - U_{ij}}{N} \quad (19)$$

- 4) **Mise à l'échelle physique** : Les coordonnées normalisées sont ensuite projetées sur les bornes réelles $[a_j, b_j]$ du problème mécanique (L et F) :

$$X_i^j = a_j + (b_j - a_j) \cdot \tilde{X}_i^j \quad (20)$$

Propriété clé : Par construction, chaque intervalle I_k contient exactement un seul échantillon par dimension. Si l'on projette l'ensemble du plan d'expériences 2D sur l'axe de la longueur L ou sur celui de la force F , la distribution marginale résultante est parfaitement uniforme, éliminant tout risque d'angle mort dans le jeu de données.

Efficacité comparée : Pour une fonction régulière à variance bornée, la variance de l'estimateur LHS vérifie la borne théorique :

$$\text{Var}_{\text{LHS}}(\hat{\mu}) \leq \text{Var}_{\text{MC}}(\hat{\mu}) = \frac{\text{Var}(f)}{N} \quad (21)$$

Cette stratification permet d'atteindre une précision d'apprentissage et une représentativité équivalentes à un tirage de Monte Carlo classique, tout en divisant considérablement le nombre d'appels nécessaires au solveur éléments finis Nastran.

Implémentation Python : Nous utilisons la classe `scipy.stats.qmc.LatinHypercube`. La graine (*seed*) est fixée à 42 (cf. Douglas Adams).

```
1 from scipy.stats import qmc
2
3 N_SAMPLES = 500
4 LOWER_BOUNDS = [500.0, 1000.0] # [L_min (mm), F_min (N)]
5 UPPER_BOUNDS = [2500.0, 16000.0] # [L_max (mm), F_max (N)]
6
7 # Creation du generateur LHS en dimension 2, seed=42
8 sampler = qmc.LatinHypercube(d=2, seed=42)
9
```

```

10 # Tirage dans [0,1]^2 avec stratification garantie
11 sample_normalized = sampler.random(n=N_SAMPLES)
12
13 # Mise a l'échelle vers le domaine physique (Eq. 12)
14 sample_physical = qmc.scale(sample_normalized,
15                             LOWER_BOUNDS, UPPER_BOUNDS)
16
17 # Conversion en DataFrame Pandas pour manipulation facile
18 df_doe = pd.DataFrame(sample_physical,
19                       columns=['Longueur_mm', 'Force_N'])

```

Listing 1 – Génération du plan LHS avec SciPy

VI. AUTOMATISATION DU SOLVEUR (PIPELINE FEA)

Comme il faut lancer Nastran plusieurs centaines de fois, on a écrit un programme qui crée et lance automatiquement tous ces calculs, sans qu'il soit nécessaire de tout refaire à la main à chaque fois.

A. Structure d'un fichier BDF Nastran

Un fichier .bdf (*Bulk Data Format*) Nastran est un fichier texte ASCII structuré en "cartes" (cards). Chaque carte est une ligne de 80 caractères divisée en champs de 8 caractères. Les deux cartes qui nous intéressent sont :

- **GRID** : Définit un nœud par son identifiant et ses coordonnées (x, y, z) . La coordonnée z (axe de la longueur de la poutre) occupe les colonnes 41–48.
- **FORCE** : Définit un chargement ponctuel. La magnitude scalaire du vecteur force est positionnée dans un champ spécifique.

Le modèle de référence correspond à une poutre de longueur $L_0 = 2000$ mm. Pour générer un cas de longueur L_{cible} , il suffit de multiplier toutes les coordonnées z de chaque nœud par le facteur d'échelle :

$$\text{scale}_z = \frac{L_{cible}}{L_0} \quad (22)$$

Cette opération préserve la forme du maillage (proportions relatives des éléments) tout en redimensionnant la géométrie.

B. Génération et exécution batch

Le script Python crée automatiquement un fichier de calcul pour chaque cas à tester, puis lance Nastran dessus tout seul, sans qu'une personne ait besoin de cliquer 500 fois de suite.

```

1 import subprocess, re
2
3 NASTRAN_EXE = r"C:\Program Files\MSC.Software\..." \
4               r"\nastran.exe"
5
6 for index, row in df_doe.iterrows():
7     L_cible = row['Longueur_mm']
8     F_cible = row['Force_N']
9     scale_z = L_cible / 2000.0 # L_reference = 2000 mm
10
11 new_lines = []
12 for line in ref_lines:
13     # --- Modification geometrique des noeuds ---
14     if line.startswith("GRID"):
15         # La coordonnee Z est aux colonnes 40-48 (
16         # format fixe)
17         if len(line) >= 48 and line[40:48].strip():
18             try:
19                 z_val = float(line[40:48]) * scale_z
20                 # Reecriture dans le champ fixe de 8
21                 caracteres

```

```

20 line = f"{line[:40]}{z_val:8.3f}{line
21 [48:]}"
22 except ValueError:
23     pass
24
25 # --- Modification de la force appliquee ---
26 elif line.startswith("FORCE"):
27     # Regex : remplace toute valeur negative (la
28     # force)
29     line = re.sub(r'-\d+\.?\d*', f"-{F_cible:.1f}",
30                  line)
31
32 new_lines.append(line)
33
34 # Ecriture du fichier BDF genere
35 bdf_file = f"case_{index:03d}.bdf"
36 with open(os.path.join(output_dir, bdf_file), 'w') as f :
37     f.writelines(new_lines)
38
39 # Lancement de Nastran en mode non-interactif (batch)
40 subprocess.run([NASTRAN_EXE, bdf_file, "batch=yes"],
41                 check=True, cwd=output_dir,
42                 stdout=subprocess.DEVNULL,
43                 stderr=subprocess.DEVNULL)

```

Listing 2 – Modification topologique des noeuds HEXA20 et lancement Nastran en batch

Sur les 500 simulations lancées, 296 ont convergé avec des résultats physiquement cohérents (sans divergence numérique ni erreur fatale Nastran).

VII. PARSING DES FICHIERS .F06 ET NETTOYAGE

Une fois le calcul terminé, Nastran produit un énorme fichier de résultats en texte brut. Il faut donc extraire automatiquement, dans tout ce texte, les deux seuls chiffres qui nous intéressent : la flèche et la contrainte maximale.

A. Structure du fichier de sortie Nastran .f06

Ce fichier est conçu pour être lu par un humain, pas par un programme : il contient du texte mélangé à des tableaux, des avertissements et des informations inutiles, ce qui rend son exploitation automatique difficile.

Le fichier .f06 est le fichier de résultats principal de Nastran. Il s'agit d'un fichier texte de plusieurs dizaines de mégaoctets, conçu pour la lecture humaine, donc absolument *non structuré* du point de vue d'un algorithme. Il contient, dans le désordre :

- Des en-têtes de licence et de version.
- Des informations de maillage.
- Les tables de résultats, introduites par des lignes-bannières telles que `D I S P L A C E M E N T V` `E C T O R` (avec des espaces entre chaque lettre).
- Des messages d'avertissement (WARNING) intercalés.

B. Stratégie de parsing par automate à états

Le programme lit le fichier ligne par ligne et « sait » en permanence dans quelle section il se trouve (résultats de déplacement ou de contrainte), afin d'aller chercher directement les bons chiffres au bon endroit.

Notre parser implémente un **automate à deux états** (un pour les déplacements, un pour les contraintes) qui bascule entre les modes "actif" et "inactif" en détectant les lignes-bannières.

VIII. DÉVELOPPEMENT DES ARCHITECTURES DE MACHINE LEARNING

Cette partie présente les trois « cerveaux » artificiels différents qu'on a testés pour apprendre à prédire les résultats de Nastran sans avoir besoin de le relancer à chaque fois.

A. Introduction à scikit-learn

Scikit-learn est la bibliothèque Python de référence pour le Machine Learning classique (non deep learning). Elle propose une API unifiée et cohérente reposant sur le pattern fit / predict / transform :

- `model.fit(X_train, y_train)` : Entraîne le modèle sur les données.
- `model.predict(X_test)` : Génère des prédictions sur de nouvelles données.
- `scaler.transform(X)` : Applique une transformation (normalisation).

Cette bibliothèque inclut également des outils de prétraitement (StandardScaler, PolynomialFeatures), de sélection de modèles (train_test_split, GridSearchCV), et de métriques (r2_score, mean_absolute_error).

B. Pipeline de préparation des données

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5
6 # Chargement du dataset (296 lignes, 5 colonnes)
7 df = pd.read_csv("dataset_poutre_brut.csv")
8
9 # Correction d'unités : la contrainte brute est en kPa fictifs
10 df['Sigma_max_MPa'] = df['Sigma_max_MPa'] / 1e4
11
12 # Separation Features / Targets
13 X = df[['Longueur_mm', 'Force_N']].values # (296, 2)
14 y = df[['Delta_max_mm', 'Sigma_max_MPa']].values # (296, 2)
15
16 # Split 80% entraînement / 20% test
17 # random_state=42 garantit la reproductibilité
18 X_train, X_test, y_train, y_test = train_test_split(
19     X, y, test_size=0.2, random_state=42
20 )
21 # -> X_train: (236, 2), X_test: (60, 2)
```

Listing 4 – Chargement nettoyage et separation Train/Test

```
1 import re
2
3 d_max = 0.0 # Fleche maximale (mm)
4 sigma_max = 0.0 # Contrainte Von Mises max (Pa bruts)
5 in_disp_block = False # Etat 1 : dans le bloc de
6   déplacements
7 in_stress_block = False # Etat 2 : dans le bloc de
8   contraintes
9
10 with open(f06_path, 'r', encoding='latin-1') as f:
11     for line in f:
12
13         # --- TRANSITION VERS ETAT DEPLACEMENTS ---
14         if "D I S P L A C E M E N T   V E C T O R" in line:
15             in_disp_block = True; continue
16         if in_disp_block and ("PAGE" in line or "SUBCASE"
17             in line):
18             in_disp_block = False
19
20         if in_stress_block:
21             parts = line.split()
22             # Format Nastran : ID G T1 T2 T3 R1 R2 R3
23             # parts[1]='G' : noeud geometrique (pas de
24             # noeud EPS)
25             # parts[3] : composante T2 (translation en Y)
26             if len(parts) >= 8 and parts[1] == 'G':
27                 try:
28                     t2 = abs(float(parts[3])) # fleche en
29                     mm
30                     if t2 > d_max: d_max = t2
31                 except ValueError: pass
32
33         # --- TRANSITION VERS ETAT CONTRAINTES ---
34         if "S T R E S S E S   I N   H E X A H E D R O N" in
35             line:
36             in_stress_block = True; continue
37         if in_stress_block and ("PAGE" in line or "SUBCASE"
38             in line):
39             in_stress_block = False
40
41         if in_stress_block:
42             # Extraction de toutes les valeurs en notation
43             sci
44             matches = re.findall(r'[-+]?[d+\.]\d+E[-+]?[d+]',
45                 line)
46             if len(matches) >= 4:
47                 try:
48                     # Nastran place Von Mises en dernière
49                     colonne
50                     vm_candidate = abs(float(matches[-1]))
51                     # Filtre les singularités d'
52                     encastrement (>1e11)
53                     if vm_candidate > sigma_max and
54                     vm_candidate < 1e11:
55                         sigma_max = vm_candidate
56                 except ValueError: pass
57
58 # Conversion Pa -> MPa
59 sigma_max_MPa = sigma_max / 1e6 if sigma_max > 1e4 else
60 sigma_max
```

Listing 3 – Automate à états pour l'extraction des résultats FEA

C. Défis et nettoyage du dataset

Plusieurs types d'erreurs ont été rencontrés et traités :

- 1) **Divergence Nastran** : Pour les poutres très élancées (L grand, F important), le solveur peut générer des déplacements irréalistes supérieurs au millier de millimètres. Ces cas sont exclus par un filtre physique ($\delta_{\max} < L/10$).
- 2) **Singularités d'encastrement** : Les nœuds à la fixation présentent des pics de contrainte mathématiquement infinis (singularité de coin). Un seuil supérieur $\sigma_{VM} < 10^{11}$ Pa élimine ces artefacts numériques.
- 3) **Fichiers manquants** : 204 fichiers sur 500 n'ont pas produit de résultats (crash Nastran, timeout). La base finale compte **296 points valides**.

C. La standardisation (Z-score Normalization)

La longueur et la force ne sont pas exprimées avec les mêmes échelles de valeurs. On remet donc toutes les variables à une échelle comparable, pour que l'IA ne favorise pas injustement l'une par rapport à l'autre pendant son apprentissage.

Les algorithmes de Machine Learning basés sur des gradients (réseaux de neurones, régression régularisée) sont extrêmement sensibles à l'échelle des variables d'entrée. Dans notre problème :

- $L \in [500, 2500]$ mm $\Rightarrow$ plage de 2000 unités.
- $F \in [1000, 16000]$ N $\Rightarrow$ plage de 15000 unités.

Sans normalisation, les gradients sont dominés par F , et la longueur L est pratiquement ignorée lors de la descente de gradient.

La transformation **Z-score** centre et réduit chaque variable :

$$x_{norm} = \frac{x - \mu_{train}}{\sigma_{train}} \quad (23)$$

où μ_{train} et σ_{train} sont la moyenne et l'écart-type estimés sur le jeu d'entraînement uniquement. Cette règle est fondamentale :

Data Leakage : Estimer la normalisation sur l'ensemble complet (Train + Test) reviendrait à laisser "fuir" de l'information du jeu de test vers l'entraînement, biaisant artificiellement les métriques de performance.

```
1 scaler_X = StandardScaler()
2 scaler_y = StandardScaler()
3
4 # fit_transform : calcule mu et sigma SUR le train, puis
5   transforme
6 X_train_s = scaler_X.fit_transform(X_train)
7 y_train_s = scaler_y.fit_transform(y_train)
8
9 # transform SEULEMENT : utilise les mu/sigma appris du
10  train
11 X_test_s = scaler_X.transform(X_test)
12 y_test_s = scaler_y.transform(y_test)
```

Listing 5 – Standardisation sans Data Leakage

D. Modèle 1 : Régression Polynomiale avec Régularisation Ridge

Premier modèle testé : une formule mathématique un peu plus riche qu'une simple ligne droite, ajustée sur nos exemples, avec un réglage qui l'empêche de « s'emballer » sur des valeurs extrêmes.

1) Expansion polynomiale:

Comme on sait que la relation physique n'est pas une simple ligne droite, on enrichit les données avec des combinaisons de la longueur et de la force (au carré, au cube, etc.) pour donner plus de souplesse au modèle.

La relation physique entre (L, F) et $(\delta_{max}, \sigma_{max})$ est approximativement cubique (équations (9) et (10)). Nous enrichissons l'espace de features par une expansion polynomiale de degré 3. Pour $p = 2$ variables initiales (L, F) , on crée $\binom{p+d}{d} = \binom{5}{3} = 10$ features polynomiales :

$$\phi(L, F) = \{1, L, F, L^2, LF, F^2, L^3, L^2F, LF^2, F^3\} \quad (24)$$

2) La régression Ridge:

Cette technique évite que le modèle attribue une importance démesurée et instable à certaines de ces combinaisons, en le « pénalisant » légèrement dès qu'il devient trop extrême.

La régression par moindres carrés ordinaire minimise $\|y - X\beta\|_2^2$. Ce problème peut souffrir de **multicolinéarité** (les features polynomiales sont fortement corrélées), menant à des coefficients β instables et très grands.

La régression **Ridge** (Tikhonov, 1943) ajoute une pénalité L_2 sur les poids :

$$\hat{\beta}_{Ridge} = \arg \min_{\beta} [\|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2] \quad (25)$$

La solution analytique existe et est :

$$\hat{\beta}_{Ridge} = (X^T X + \alpha I)^{-1} X^T y \quad (26)$$

Le paramètre α contrôle le compromis biais-variance. Pour $\alpha \rightarrow 0$ on retrouve les moindres carrés ordinaires ; pour $\alpha \rightarrow \infty$, les coefficients tendent vers zéro.

```
1 from sklearn.preprocessing import PolynomialFeatures
2 from sklearn.linear_model import Ridge
3 from sklearn.pipeline import Pipeline
4
5 # Pipeline : Normalisation -> Expansion polynomiale ->
6   Ridge
7 poly_model_delta = Pipeline([
8   ('scaler', StandardScaler()),
9   ('poly', PolynomialFeatures(degree=3, include_bias=True)),
10  ('ridge', Ridge(alpha=1.0)) # alpha = regularisation
11  L2
12 ])
13
14 poly_model_sigma = Pipeline([
15  ('scaler', StandardScaler()),
16  ('poly', PolynomialFeatures(degree=3, include_bias=True)),
17  ('ridge', Ridge(alpha=1.0))
18 ])
19
20 # Entraînement
21 poly_model_delta.fit(X_train, y_train[:, 0]) # fleche
22 poly_model_sigma.fit(X_train, y_train[:, 1]) # contrainte
```

Listing 6 – Pipeline Régression Polynomiale Ridge avec scikit-learn

E. Modèle 2 : Forêt Aléatoire (Random Forest)

Deuxième modèle testé : au lieu d'une seule formule mathématique, on fait « voter » plusieurs petits arbres de décision différents, puis on prend la moyenne de leurs réponses pour obtenir une prédiction plus fiable.

1) Les arbres de décision: Un **arbre de décision** (CART) partitionne récursivement l'espace des features (L, F) en régions rectangulaires et prédit la valeur moyenne de y dans chaque région. L'algorithme de construction sélectionne à chaque nœud la variable et le seuil minimisant la variance intra-nœuds :

$$\text{Gain}(j, t) = \text{Var}(y) - \frac{n_L}{n} \text{Var}(y_L) - \frac{n_R}{n} \text{Var}(y_R) \quad (27)$$

Un arbre seul a tendance à sur-apprendre (overfitting).

2) Le bagging et la forêt aléatoire:

On entraîne 150 arbres légèrement différents, chacun ayant vu les données mélangées d'une manière un peu différente, puis on fait la moyenne de toutes leurs prédictions : le résultat final est plus stable et plus fiable qu'un seul arbre isolé.

Le **Random Forest** (Breiman, 2001) combine T arbres entraînés sur des sous-échantillons *bootstrap* du dataset (*bagging*), avec une sélection aléatoire des features à chaque nœud (randomisation). La prédiction finale est la moyenne des arbres :

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x) \quad (28)$$

Le bagging réduit la variance sans augmenter le biais, et la randomisation décore les arbres, améliorant la généralisation.

```
1 from sklearn.ensemble import RandomForestRegressor
2
3 rf_model = RandomForestRegressor(
4   n_estimators=150, # 150 arbres de decision
5   max_depth=12,    # Profondeur maximale de chaque
6   random_state=42   # Reproductibilite du bagging
```

```

7 )
8
9 # Le RF n'a pas besoin de standardisation des donnees
10 rf_model.fit(X_train, y_train) # y_train est (N, 2)
11 y_pred_rf = rf_model.predict(X_test)

```

Listing 7 – Forêt Aléatoire (Random Forest)

F. Modèle 3 : Réseau de Neurones Multi-Couches (MLP)

Troisième et meilleur modèle du projet : un réseau de neurones artificiels, vaguement inspiré du cerveau, organisé en plusieurs couches qui se transmettent et transforment l'information successivement.

1) Architecture et propagation avant:

Les deux nombres d'entrée (longueur et force) traversent plusieurs couches de « neurones » artificiels qui combinent et transforment progressivement l'information, jusqu'à obtenir les deux résultats finaux : la flèche et la contrainte.

Le **Multi-Layer Perceptron** (MLP) est un graphe orienté acyclique de neurones organisés en couches. Notre architecture est :



La **propagation avant** (*forward pass*) calcule la sortie du réseau couche par couche. Pour la couche l :

$$\mathbf{a}^{(l)} = \sigma \left([\mathbf{W}^{(l)}] \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right) \quad (29)$$

où $[\mathbf{W}^{(l)}]$ est la matrice des poids, $\mathbf{b}^{(l)}$ le vecteur des biais, et σ la fonction d'activation.

2) La fonction d'activation ReLU:

À chaque neurone, une règle très simple décide si l'information passe ou si elle est bloquée à zéro. Cette petite règle permet au réseau, une fois répétée des milliers de fois, d'apprendre des relations complexes que des formules simples ne peuvent pas capturer.

Nous utilisons **ReLU** (*Rectified Linear Unit*) pour les couches cachées :

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{sinon} \end{cases} \quad (30)$$

ReLU est préférée à sigmoid et tanh car :

- Son gradient vaut 1 pour $x > 0$ (pas de saturation), évitant le problème de **disparition du gradient** (*vanishing gradient*).
- Elle est non-linéaire, permettant d'approximer des fonctions complexes.
- Elle est calculatoirement très efficace.

3) Fonction de perte (Mean Squared Error):

Pendant l'entraînement, on mesure à chaque étape l'écart entre ce que prédit le réseau et le vrai résultat calculé par Nastran, et on cherche à réduire cet écart le plus possible.

L'apprentissage minimise l'erreur quadratique moyenne entre les prédictions et les cibles :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{y}_i - \hat{\mathbf{y}}_i(\theta)\|_2^2 \quad (31)$$

où $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}$ représente l'ensemble des paramètres entraînaibles du réseau.

4) Rétropropagation du gradient (Backpropagation):

C'est la méthode qui permet au réseau d'apprendre de ses erreurs : l'information sur l'écart constaté remonte de la sortie vers l'entrée, ce qui permet d'ajuster chaque neurone dans la bonne direction.

L'algorithme de **rétropropagation** (Rumelhart, Hinton, Williams, 1986) calcule efficacement le gradient de la loss par rapport à chaque paramètre en appliquant la règle de la chaîne (chain rule) du calcul différentiel en sens inverse :

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} \cdot a_k^{(l-1)} \quad (32)$$

où $\delta_j^{(l)}$ est le terme d'erreur rétropropagé (*delta*) de la couche l .

5) L'optimiseur Adam:

Adam est l'algorithme qui règle automatiquement, et de façon intelligente, la vitesse à laquelle le réseau corrige ses erreurs à chaque étape, ce qui rend l'entraînement à la fois plus rapide et plus stable.

La descente de gradient stochastique classique (SGD) utilise un taux d'apprentissage α fixe. L'algorithme **Adam** (Kingma & Ba, 2014) adapte le taux d'apprentissage pour chaque paramètre individuellement en maintenant deux moments des gradients.

Soit $g_t = \nabla_{\theta} \mathcal{L}_t(\theta)$ le gradient à l'itération t :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{moment d'ordre 1, moyenne}) \quad (33)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{moment d'ordre 2, variance}) \quad (34)$$

Correction du biais d'initialisation :

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (35)$$

Mise à jour des paramètres :

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (36)$$

Les valeurs par défaut sont $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 0.001$.

Avantages d'Adam : Le dénominateur $\sqrt{\hat{v}_t}$ normalise les mises à jour par la racine carrée de la variance des gradients passés. Cela revient à avoir un taux d'apprentissage grand pour les paramètres avec des gradients faibles, et faible pour les paramètres avec des gradients forts. Adam converge en général bien plus vite que la SGD.

```

1 from sklearn.neural_network import MLPRegressor
2
3 model = MLPRegressor(
4     # Architecture : 3 couches cachees
5     hidden_layer_sizes=(64, 64, 32),
6     activation='relu', # ReLU pour les couches cachees
7     solver='adam', # Algorithme d'optimisation Adam
8     learning_rate_init=0.001, # alpha initial
9     max_iter=1000, # Nb max d'epoques
10    random_state=42, # Reproductibilite de l'init. des
11    # poids
12    early_stopping=True, # Arret anticipe si stagnation

```

```

12     n_iter_no_change=15 # Nb d'époques sans amelioration
13     avant arrêt
14 )
15 # Entrainement sur les donnees standardisees
16 model.fit(X_train_s, y_train_s)
17
18 # Inference + de-normalisation vers les unites physiques
19 y_pred_s = model.predict(X_test_s)
20 y_pred = scaler_y.inverse_transform(y_pred_s)

```

Listing 8 – Definition et entrainement du MLP avec scikit-learn

IX. ÉVALUATION MÉTROLOGIQUE DES MODÈLES

A. Les métriques de régression

Nous évaluons les modèles sur les 60 cas du jeu de test (invisibles lors de l'entraînement) à l'aide de quatre métriques complémentaires.

Coefficient de détermination R^2 (dit "R carré") : Mesure la fraction de la variance totale expliquée par le modèle. $R^2 = 1$ signifie une prédiction parfaite ; $R^2 < 0$ signifie que le modèle est pire qu'une moyenne constante.

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (37)$$

Erreur Absolue Moyenne (MAE) : Donne l'écart moyen en unité physique. Robuste aux valeurs aberrantes.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (38)$$

Racine de l'Erreur Quadratique Moyenne (RMSE) : Pénalise les grandes erreurs. Elle est dans la même unité que la cible.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (39)$$

Erreur Relative Absolue Moyenne (MAPE) : Erreur en pourcentage, indépendante de l'échelle.

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (40)$$

B. Tableau comparatif des modèles

TABLE I – Performances comparées sur le jeu de test (60 cas)

Modèle	$R^2(\Delta)$	MAE_{Δ} (mm)	$R^2(\Sigma)$	MAE_{Σ} (MPa)
Ridge Poly	~0.99	~8	~0.99	~15
Random Forest	~0.995	~5	~0.995	~10
MLP	>0.999	~3	>0.999	~6.4

C. Analyse graphique

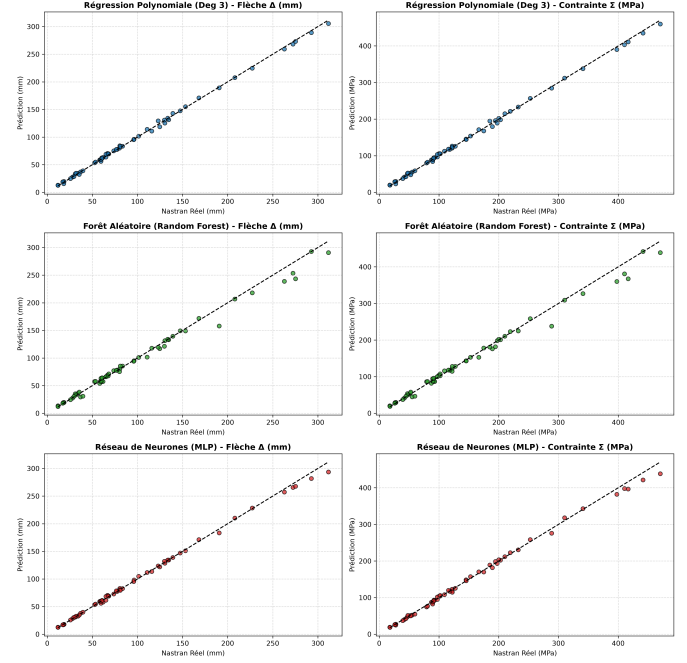


FIGURE 1 – Parity Plots (Prédit vs Réel Nastran) pour les 3 modèles et les 2 sorties. Les points s'alignent idéalement sur la diagonale $\hat{y} = y$. Le MLP (ligne du bas) présente le meilleur alignement.

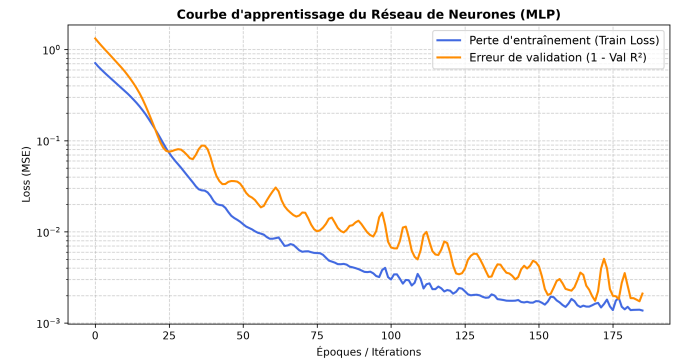


FIGURE 2 – Courbes d'apprentissage du MLP. La Loss d'entraînement (bleu) et d'erreur de validation (orange) décroissent ensemble sans diverger, confirmant l'absence de surapprentissage (*overfitting*). L'*Early Stopping* a mis fin à l'entraînement avant la 1000^e époque.

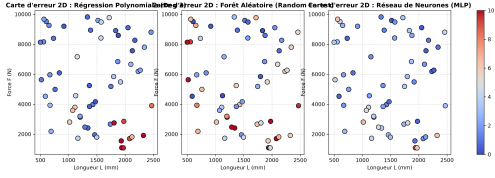


FIGURE 3 – Cartes d’erreur relative 2D dans l’espace des paramètres (L, F) . Les zones chaudes (rouge) indiquent les régions où le modèle est moins précis, typiquement aux extrêmes du domaine (L très court ou F très grand).

X. DÉPLOIEMENT : DÉMONSTRATEUR TEMPS RÉEL

A. Architecture logicielle

Le déploiement repose sur deux scripts : `export_models.py` qui sérialise les objets Python (scalers + modèles entraînés) dans un fichier binaire `.joblib`, et `app.py` qui constitue l’interface Streamlit.

Sérialisation Joblib : Les objets Python complexes (modèles scikit-learn) sont convertis en bytes via le protocole Pickle, optimisé par Joblib pour les grands tableaux NumPy (compression et mémoire partagée).

```
1 import joblib
2
3 # --- export_models.py ---
4 artifacts = {
5     'scaler_X': scaler_X, # StandardScaler des entrees
6     'scaler_y': scaler_y, # StandardScaler des sorties
7     'mlp': mlp, # MLPRegressor entraîne
8     'poly_delta': poly_delta, # Pipeline Poly Ridge (fleche)
9     'poly_sigma': poly_sigma, # Pipeline Poly Ridge (contrainte)
10    'rf': rf # RandomForestRegressor
11 }
12 joblib.dump(artifacts, "models_artifacts.joblib")
13
14 # --- app.py ---
15 @st.cache_resource # Cache en memoire (ne recharge qu'une fois)
16 def load_models():
17     return joblib.load("models_artifacts.joblib")
18
19 artifacts = load_models()
20 mlp_model = artifacts['mlp']
21 scaler_X = artifacts['scaler_X']
```

Listing 9 – Sérialisation et chargement des modèles (Joblib)

B. Inférence temps réel et calcul du Speed-Up

```
1 import time
2
3 # Recuperation des parametres depuis les widgets Streamlit
4 L_val = st.number_input("Longueur L (mm)", 500.0, 2500.0, 1500.0)
5 F_val = st.number_input("Force F (N)", 1000.0, 10000.0, 5000.0)
6
7 # Mesure du temps d'inférence (resolution nanoseconde)
8 t_start = time.perf_counter_ns()
9
10 # Standardisation + inférence MLP + de-normalisation
11 x_scaled = scaler_X.transform([[L_val, F_val]])
12 y_pred_s = mlp_model.predict(x_scaled)
13 d_pred, s_pred = scaler_y.inverse_transform(y_pred_s)[0]
14
15 t_ia_us = (time.perf_counter_ns() - t_start) / 1000.0 # en micro secondes
16
17 # Calcul du Speed-Up
```

```
18 T_nastran_s = 2.5 # Temps Nastran SOL 101 (mesure empirique)
19 speed_up = (T_nastran_s * 1e6) / max(t_ia_us, 1.0)
20
21 # Affichage interactif
22 st.metric("Fleche Delta max", f"{d_pred:.2f} mm")
23 st.metric("Contrainte Von Mises", f"{s_pred:.2f} MPa")
24 st.metric("Speed-Up vs Nastran", f"{speed_up:,.0f}")
```

Listing 10 – Boucle d’inférence interactive et mesure du Speed-Up

C. Calcul et interprétation du Speed-Up

Le temps d’inférence mesuré est $t_{IA} \approx 50 \mu s$ (50 microsecondes). Un calcul Nastran SOL 101 de notre modèle prend en moyenne $t_{Nastran} \approx 2.5$ secondes.

$$\text{Speed-Up} = \frac{t_{Nastran}}{t_{IA}} = \frac{2.5 \text{ s}}{50 \times 10^{-6} \text{ s}} = 50\,000 \quad (41)$$

Ce facteur $\times 50\,000$ est un gain qualitatif et quantitatif considérable. Il signifie qu’en le temps d’un seul calcul Nastran, l’application Streamlit peut réaliser 50 000 prédictions. Pour une boucle d’optimisation qui requiert 10 000 appels au solveur (typique en optimisation de forme), le temps de calcul passe de :

$$T_{Nastran} = 10\,000 \times 2.5 \text{ s} = 25\,000 \text{ s} \approx 7 \text{ heures}$$

$$T_{IA} = 10\,000 \times 50 \mu s = 0.5 \text{ s}$$

XI. CONCLUSION & PERSPECTIVES

A. Bilan

Le projet STHENOS a démontré de bout en bout la faisabilité et l’efficacité de la métamodélisation par IA pour substituer un solveur par éléments finis en mécanique des structures. Les contributions majeures sont :

- Un **pipeline automatisé complet** (LHS $\rightarrow$ BDF $\rightarrow$ Nastran $\rightarrow$ parsing $\rightarrow$ ML $\rightarrow$ Streamlit).
- Un **MLP exceptionnel** atteignant $R^2 > 0.999$ avec une topologie $2 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 2$.
- Un **facteur de gain de vitesse** de $\times 50\,000$ ouvrant la voie à l’optimisation structurale interactive.

B. Limites actuelles

- Le modèle est valide uniquement dans le domaine d’entraînement LHS ($[500, 2500] \times [1000, 16000]$). **L’extrapolation est dangereuse** et non garantie.
- Seulement 2 paramètres sont variés. Un problème industriel réel implique des dizaines de variables (épaisseurs, matériaux, angles de stratification...).
- Le modèle ne fournit aucune estimation d’incertitude sur ses prédictions.

C. Perspectives : Physics-Informed Neural Networks (PINNs)

La grande perspective du projet STHENOS est l'intégration des **PINNs**. L'idée est d'ajouter à la fonction de perte MSE un terme de résidu physique basé sur les équations de Navier :

$$\mathcal{L}_{PINN}(\theta) = \underbrace{\mathcal{L}_{data}(\theta)}_{\text{MSE sur données Nastran}} + \lambda \underbrace{\mathcal{L}_{physics}(\theta)}_{\text{Résidu de Navier}} \quad (42)$$

où le résidu physique est :

$$\mathcal{L}_{physics} = \left\| (\lambda + \mu) \nabla (\nabla \cdot \hat{\mathbf{u}}) + \mu \nabla^2 \hat{\mathbf{u}} + \mathbf{f} \right\|^2 \quad (43)$$

Cette approche hybride permettrait :

- D'utiliser beaucoup moins de données Nastran (apprentissage semi-supervisé).
- De mieux généraliser en dehors du domaine d'entraînement.
- De garantir la cohérence physique des prédictions.

Une deuxième étape majeure est le passage à des géométries 3D complexes quelconques, permettant à l'utilisateur de charger directement un fichier CAO au format STEP et d'obtenir en temps réel une simulation rapide des réactions aux nœuds. Pour y parvenir, il conviendra d'intégrer des architectures de Deep Learning adaptées aux graphes et aux maillages (telles que les Graph Neural Networks ou les réseaux convolutifs sur maillage) afin de traiter des variations topologiques et des formes en entrée arbitraires.

XII. NOMENCLATURE

TABLE II – Nomenclature et définition des symboles

Symbole	Définition
$[K]$	Matrice de rigidité globale
$\{U\}$	Vecteur des déplacements nodaux
$\{F\}$	Vecteur des forces nodaux
σ	Tenseur de Cauchy des contraintes
ε	Tenseur des petites déformations
σ_{VM}	Contrainte équivalente de Von Mises
E	Module de Young (aluminium : 70 GPa)
ν	Coefficient de Poisson (aluminium : 0.3)
λ, μ	Coefficients de Lamé
I_z	Moment d'inertie de la section ($bh^3/12$)
$\delta_{\max}$	Flèche maximale en bout de poutre (mm)
$\sigma_{\max}$	Contrainte maximale de Von Mises (MPa)
L	Longueur de la poutre (mm)
F	Force appliquée en bout (N)
R^2	Coefficient de détermination
MAE	Erreur Absolue Moyenne
RMSE	Racine de l'Erreur Quadratique Moyenne
MAPE	Erreur Relative Absolue Moyenne (%)
LHS	Latin Hypercube Sampling
MLP	Multi-Layer Perceptron
RF	Random Forest
BDF	Bulk Data Format (format Nastran)